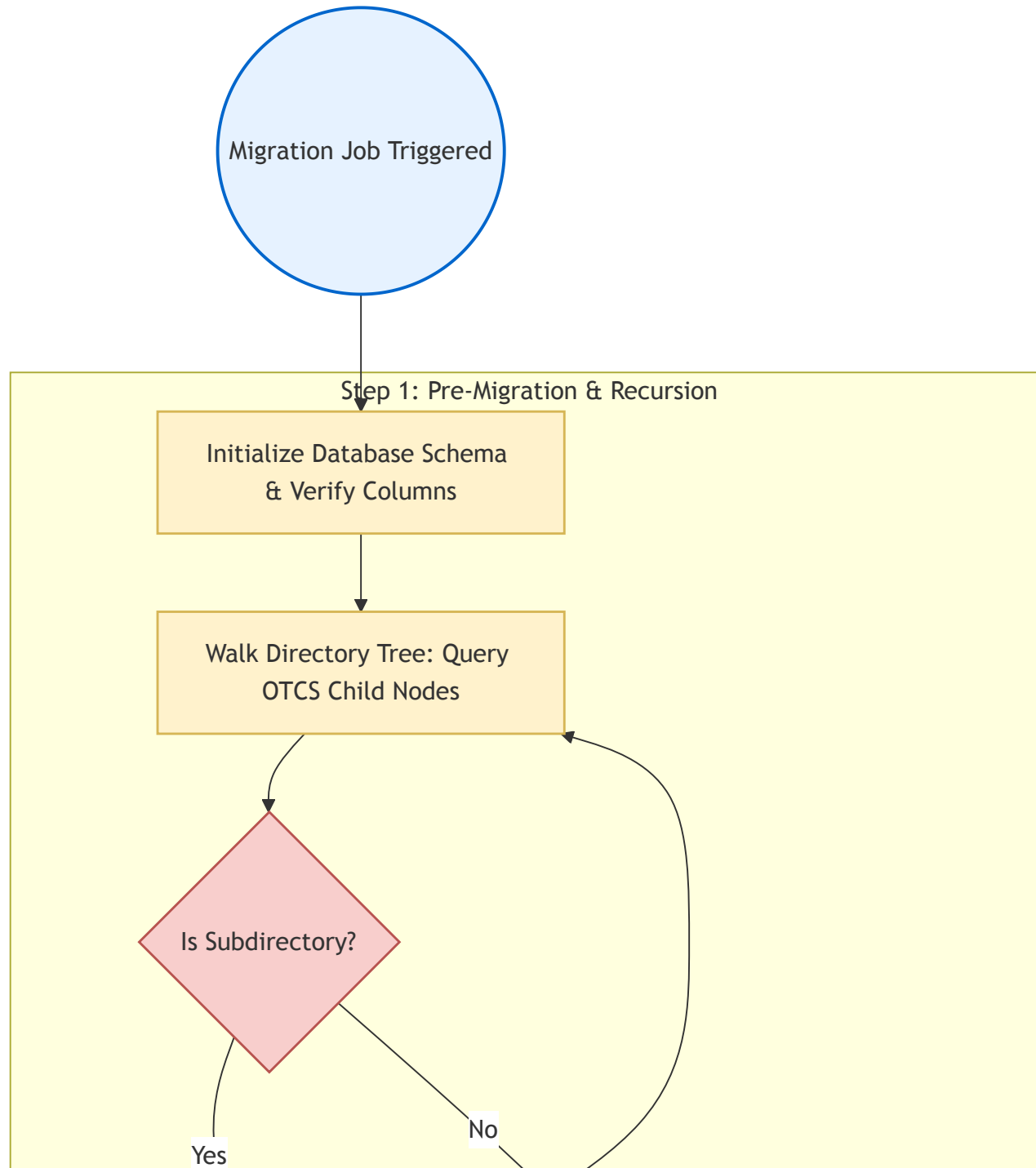


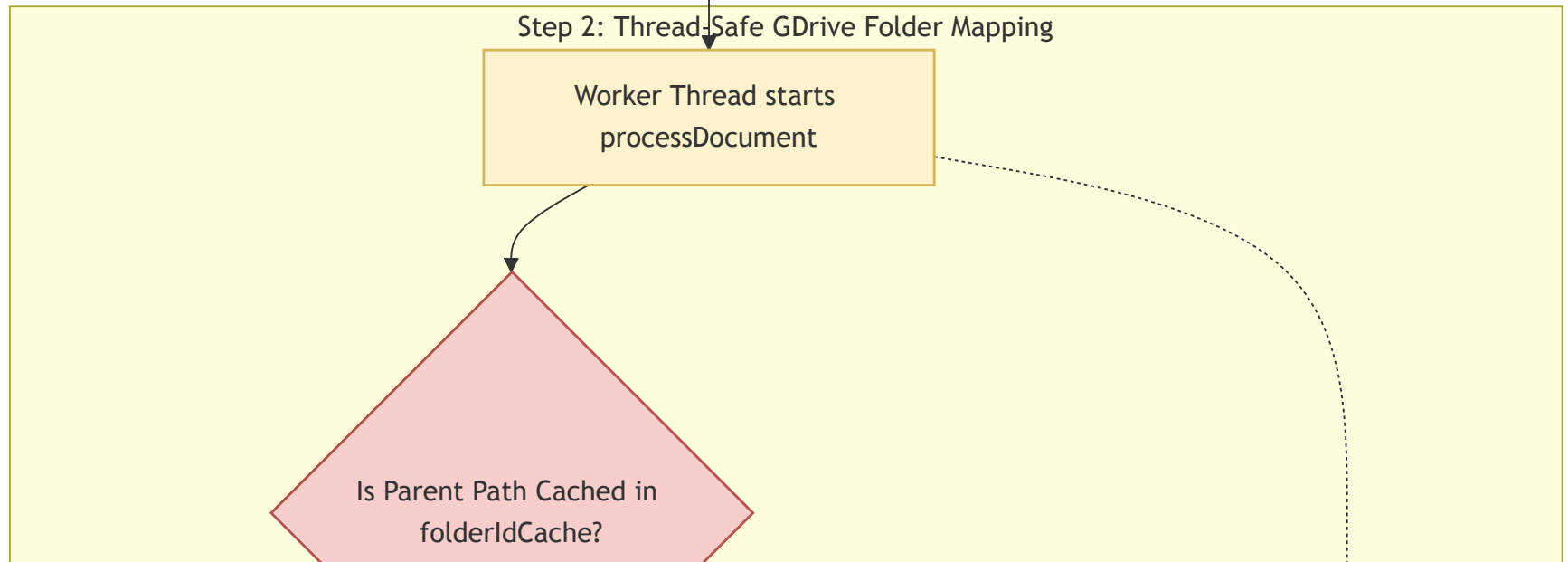
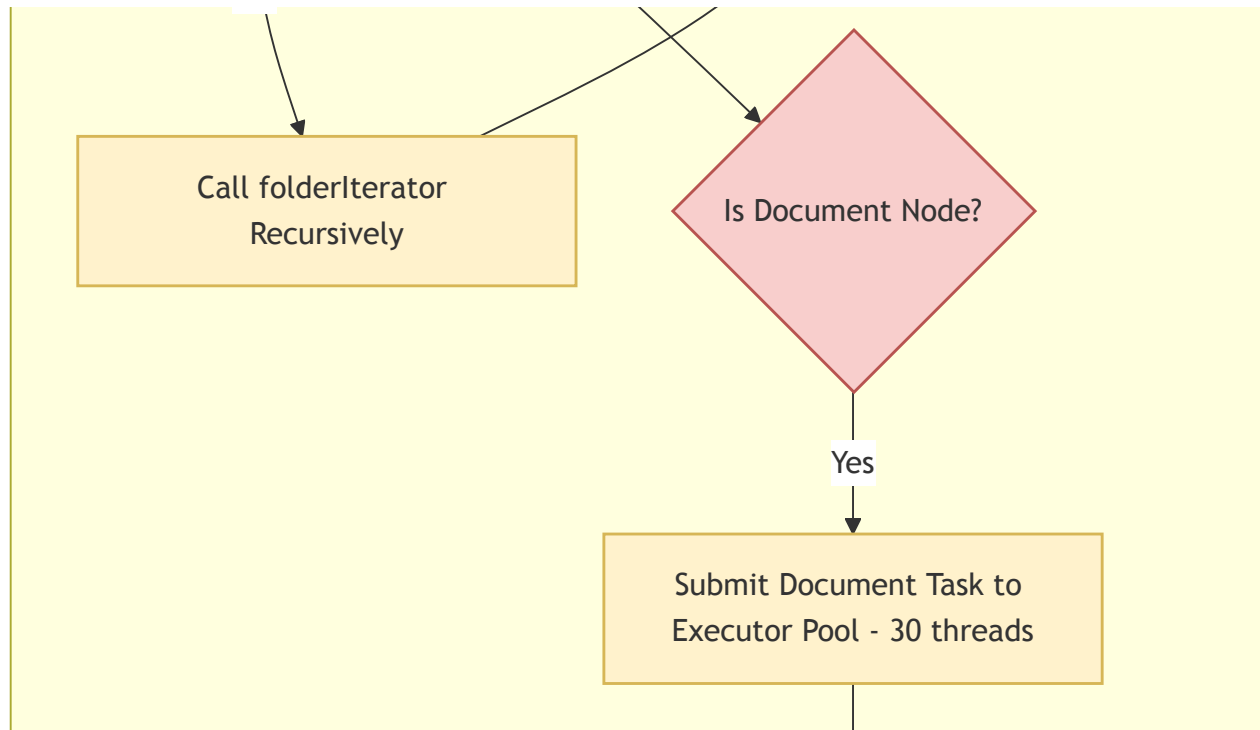
Operational Flow & Exception Handling: Batch Migration

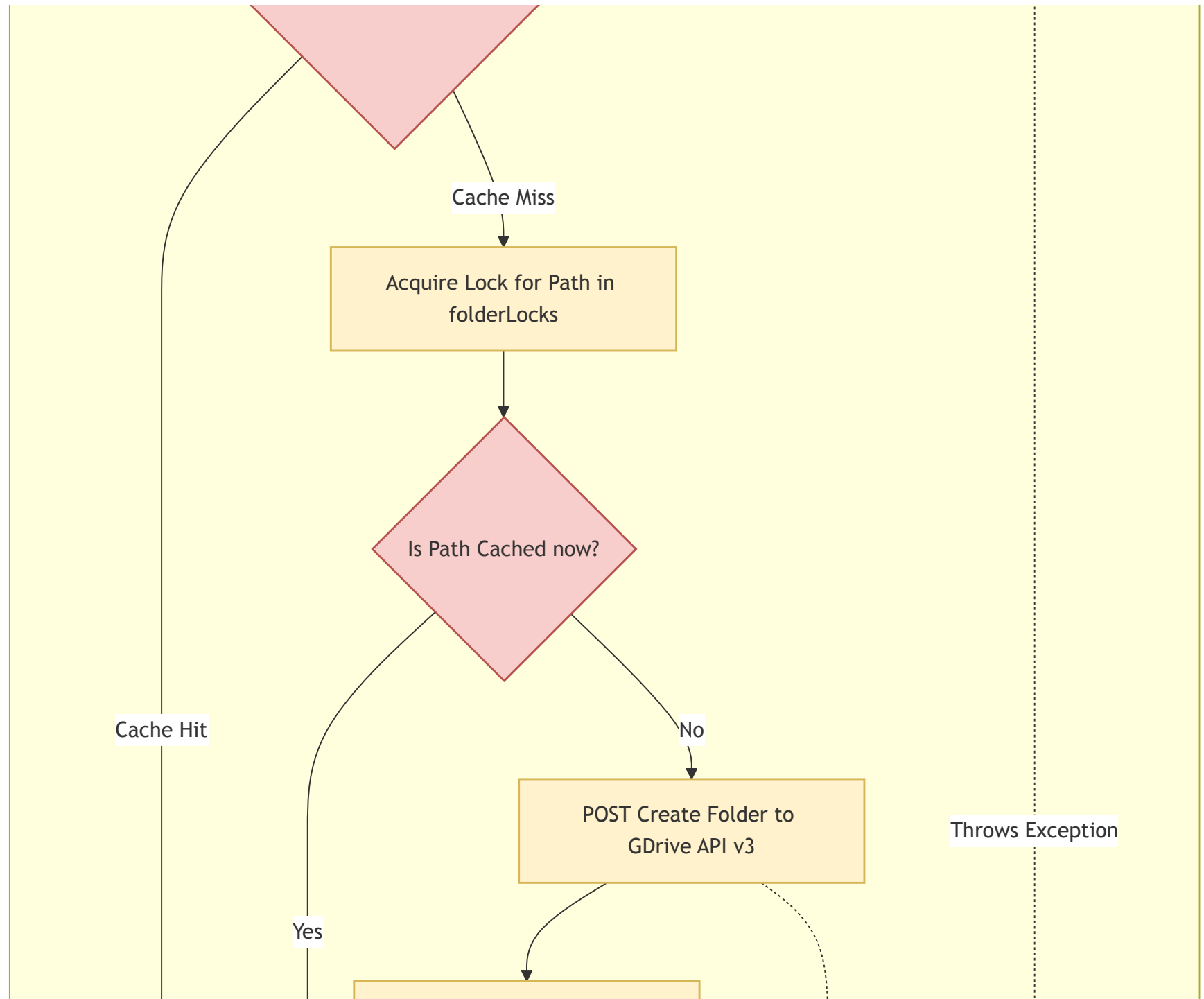
This document details the complete end-to-end execution flow and exception propagation system for the Multi-Threaded Batch Migration process.

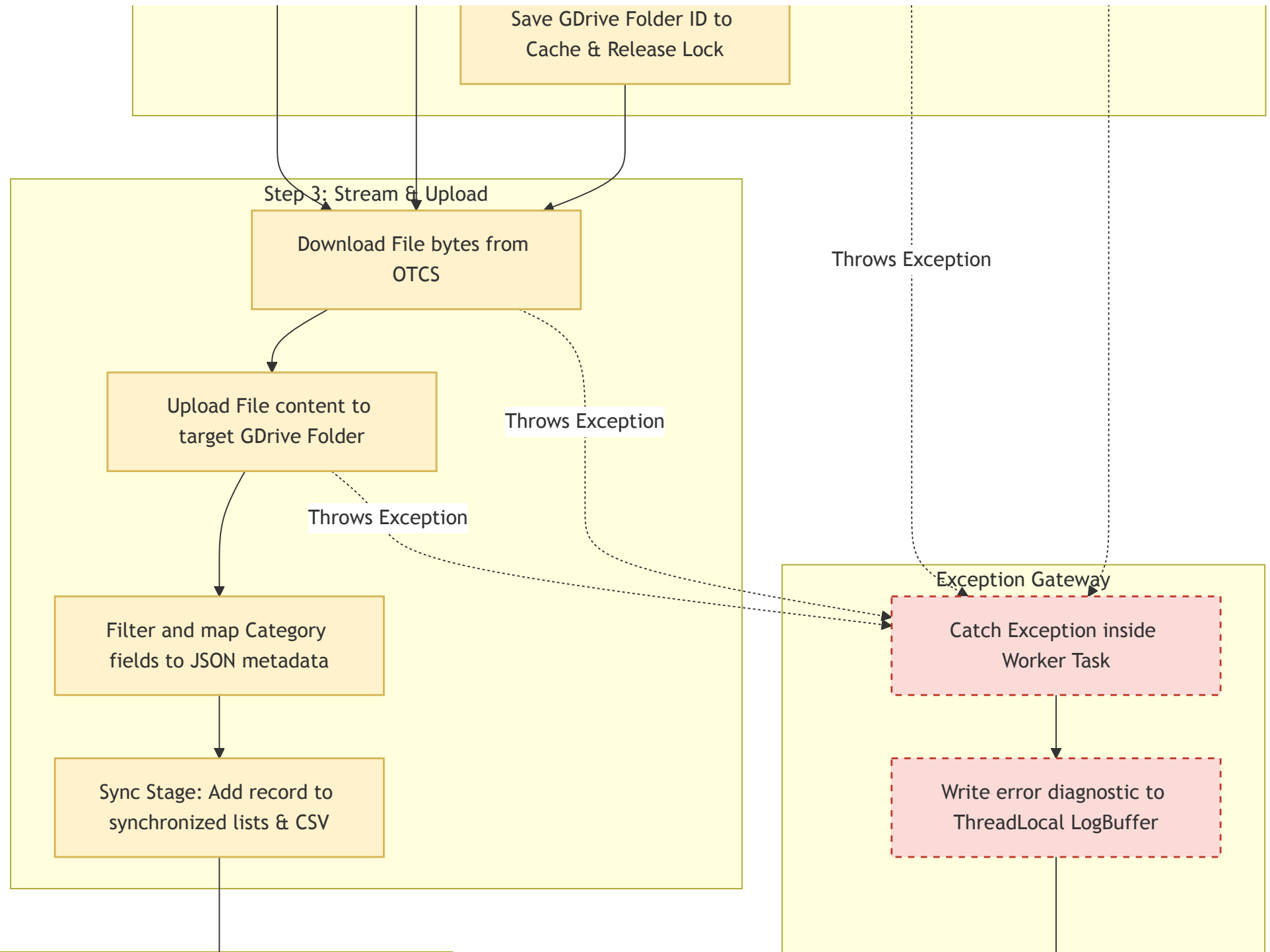
1. Process Flow Diagram (Boxes & Arrows)

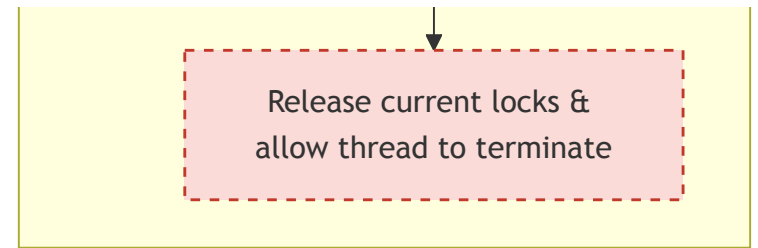
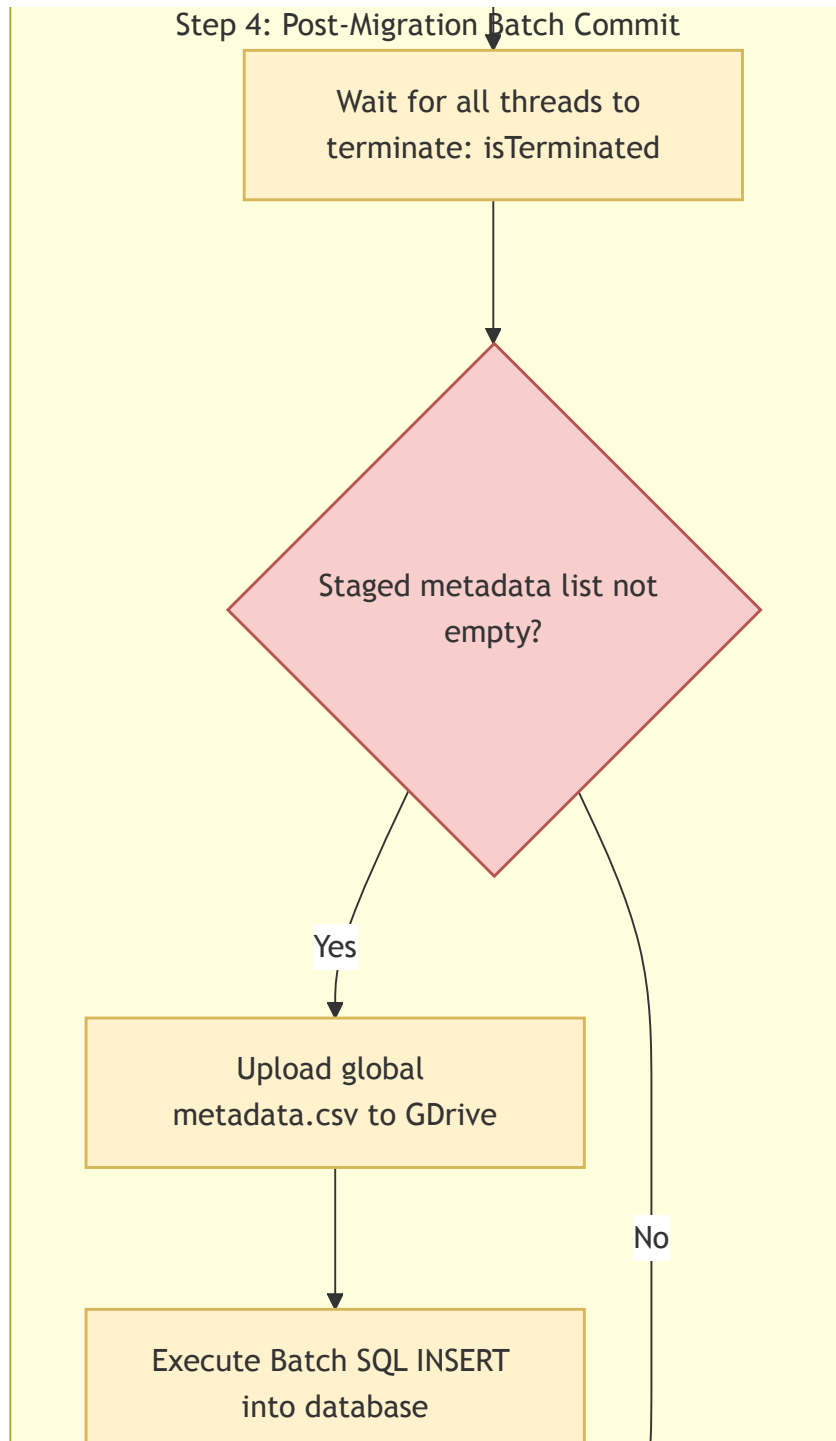
This flowchart traces the step-by-step process of the batch migration pipeline, highlighting directory recursion, thread pool scheduling, cache locking, and batch database loading.

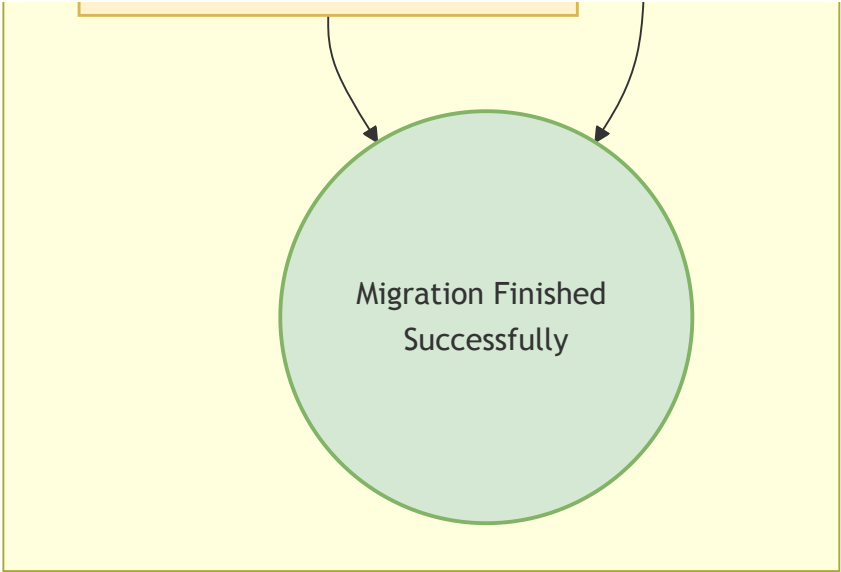




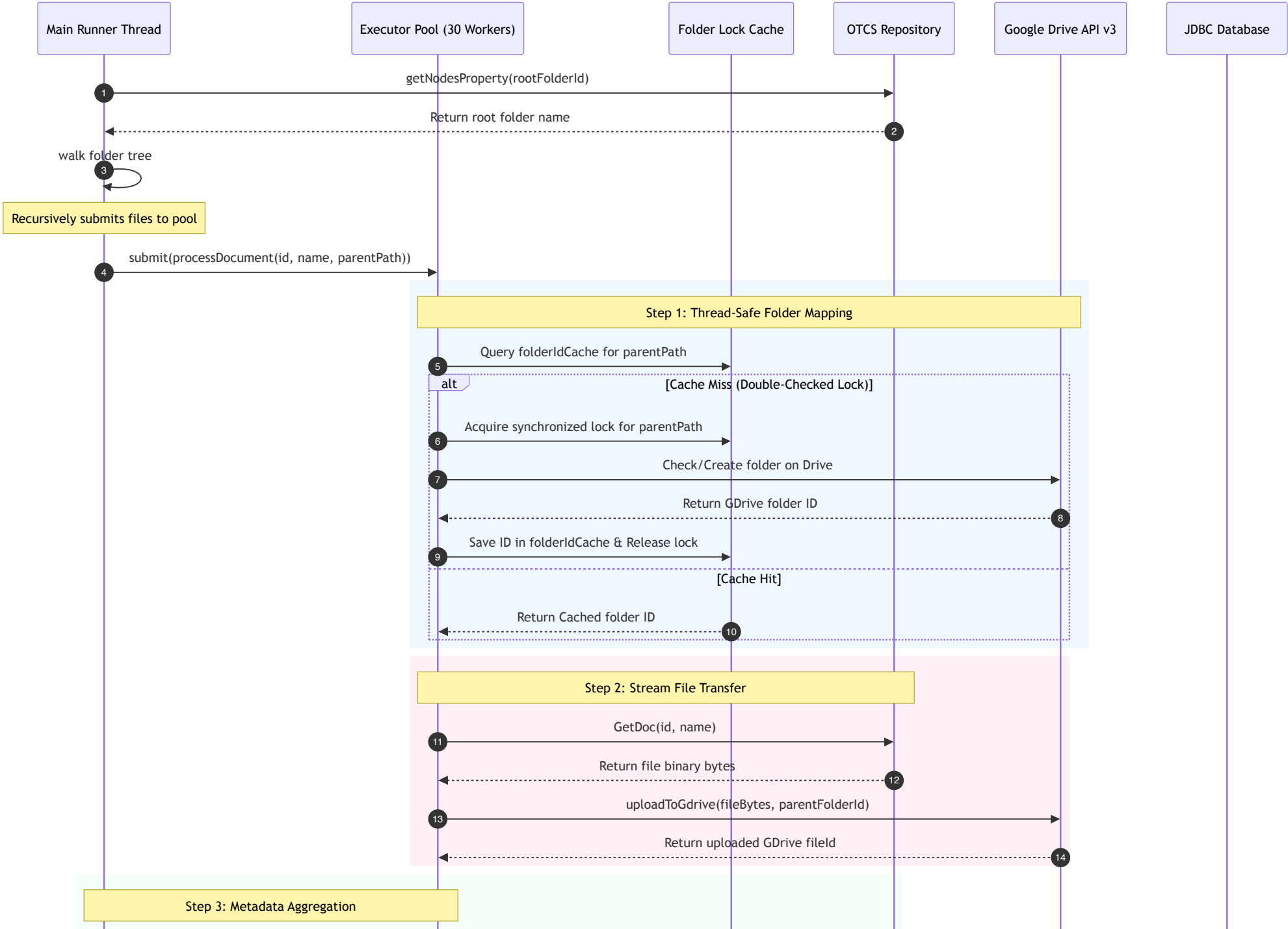


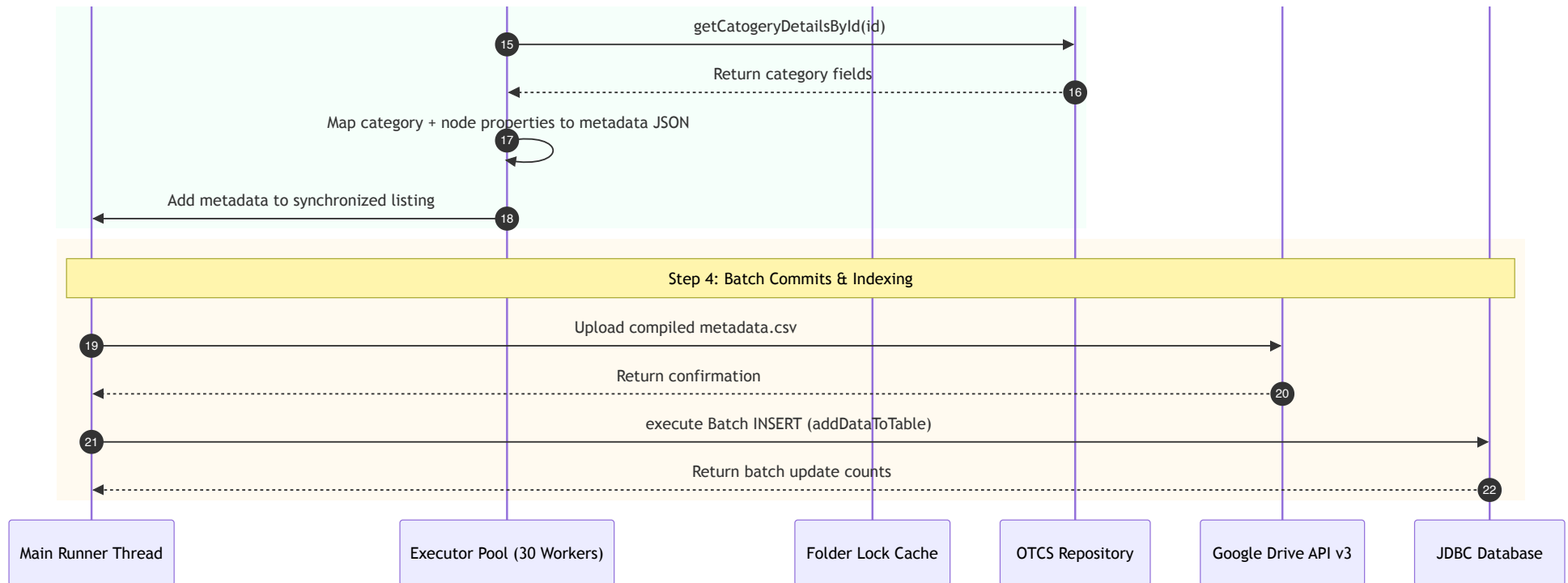






2. Happy Path Sequence Diagram





3. Step-by-Step Execution Mechanics

1. Initialization (`App.java#run`):

- Spawns the Spring Boot command-line program.
- Triggers `otcsToGdriveService.exportOtcsToGdrive()`.

2. Recursive Traversal (`otcsToGdriveService.java#folderIterator`):

- Recursively queries child nodes via `searchDocs.getChildNodes`.
- If a node is a directory, it invokes `folderIterator` recursively, maintaining a path structure (e.g. `Root/FolderA/FolderB/`).
- If a node is a file (`type == 144`), it submits a runnable task `processDocument` to the fixed thread pool (`ExecutorService` size 30).

3. Thread-Safe GDrive Folder Mappings (`otcsToGdriveService.java#processDocument`):

- Uses **Double-Checked Locking** over path structures. Checks if the folder exists in the thread-safe `folderIdCache` map.

- On cache miss, it synchronizes on a lock object specifically mapped to the directory path using `folderLocks.computeIfAbsent` .
- Checks the cache a second time inside the synchronization block. If it is still missing, it makes the Google API call to verify/create the directory, saves the folder ID to `folderIdCache` , and releases the lock.

4. File Streaming & Metadata Filtering:

- Downloads document bytes from Content Server via `downloadDoc.GetDoc` .
- Streams the bytes to GDrive via `uploadFileContentGDrive.uploadToGdrive` using pooled connections.
- Extracts categories, properties, and creator details via OpenText REST APIs.
- Maps variables (e.g., Parent ID, Path, Created By, GDrive edit URL) into a combined JSON metadata schema.
- Synchronizes on `csvContent` to append the record's metadata line, and inserts the JSON record into `jsonMetadataList` .

5. Post-Migration Batch DB Commit:

- The main thread invokes `executorService.shutdown()` and blocks in a `while` loop until `isTerminated()` returns true, confirming all migrations are complete.
- Combines the CSV content and uploads it to Google Drive as `metadata.csv` .
- Calls `dbOperation.addDataToTable(jsonMetadataList)` to perform a high-performance **SQL Batch Insert** into the database using JDBC templates, completing the migration lifecycle.

4. Exception Handling & Propagation Details

Thread-Isolated Exception Isolation

- Each `processDocument` task executes within its own thread context.
- To prevent a single document error from crashing the entire batch process, the runnable execution is fully isolated inside a `try-catch-finally` block:
 - If a file download, folder creation, or upload operation throws an exception, the thread catches the exception and prints the trace directly to a `ThreadLocal LogBuffer` .

- In the `finally` block, the log buffer writes the complete, un-interleaved sequence to the logs, clears its buffer, and allows the worker thread to exit cleanly.
- The parent migration job completes with a summary of the successfully processed files and logged exceptions.